

To include multer with the NodeJS project: `npm i --save multer uuid mime`

Then to use it:

```
const express = require('express')
const multer = require('multer')
const path = require('path')
const fs = require('fs')
const mime = require('mime')
const { v4: uuidv4 } = require('uuid')
=====
```

To define disk storage for the multer object:

```
// Ensure uploads directory exists
const uploadDir = path.join(__dirname, 'uploads')
if (!fs.existsSync(uploadDir)) {
  fs.mkdirSync(uploadDir, { recursive: true })
}

// Configure the storage
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, uploadDir)
  },
  filename: (req, file, cb) => {
    const uniqueName = `${uuidv4()}-${file.originalname}`
    cb(null, uniqueName)
  }
})
=====
```

To define the image/file filter function:

```
// File filter - ONLY allow images
const imageFilter = (req, file, cb) => {
  // Allowed mime types
  const allowedMimes = [
    'image/jpeg',
    'image/jpg',
    'image/png',
    'image/gif',
    'image/webp',
    'image/bmp',
  ];

  // Check mime type
  if (allowedMimes.includes(file.mimetype)) {
    cb(null, true); // Accept file
  } else {
    cb(new Error(`Invalid file type: ${file.mimetype}. Only images are
allowed!`), false);
  }
};
=====
```

To define the multer object that use the storage, filter function, and the limits:

```
// Configure multer with limits and filter
const upload = multer({
  storage: storage,
  fileFilter: imageFilter,
  limits: {
    fileSize: 2 * 1024 * 1024,
    files: 3,
  }
});
=====
```

To upload multiple images/files in the same request:

```
/**
 * POST /upload
 * Upload up to 3 images
 * Field name must be 'images'
 */
app.post('/upload', upload.array('images', 3), (req, res) => {
  try {
    // Check if files were uploaded
    if (!req.files || req.files.length === 0) {
      return res.status(400).json({
        success: false,
        message: 'No image files uploaded or files were rejected (non-image
types)'
      });
    }

    // Format response with file details
    const uploadedFiles = req.files.map(file => ({
      originalName: file.originalname,
      filename: file.filename,
      size: file.size,
      mimetype: file.mimetype,
      url: `/uploads/${file.filename}`
    }));

    res.status(200).json({
      success: true,
      message: `${uploadedFiles.length} image(s) uploaded successfully`,
      files: uploadedFiles
    });

  } catch (error) {
    res.status(500).json({
      success: false,
      message: 'Server error during upload',
      error: error.message
    });
  }
});
```

=====

To upload a single file:

```
/**
 * POST /upload-single
 * Upload single image (alternative endpoint)
 */
app.post('/upload-single', upload.single('image'), (req, res) => {
  try {
    if (!req.file) {
      return res.status(400).json({
        success: false,
        message: 'No image file uploaded or file was rejected'
      });
    }

    res.status(200).json({
      success: true,
      message: 'Image uploaded successfully',
      file: {
        originalName: req.file.originalname,
        filename: req.file.filename,
        size: req.file.size,
        mimetype: req.file.mimetype,
        url: `/uploads/${req.file.filename}`
      }
    });
  } catch (error) {
    res.status(500).json({
      success: false,
      message: 'Server error during upload',
      error: error.message
    });
  }
});
```

=====

To get the image/file from the storage:

```
app.get('/get-image/:imagename', (req, res) => {
  const imagePath = `${uploadDir}/${req.params['imagename']}`
  // console.log(`imagePath: ${imagePath}`)

  if(fs.existsSync(imagePath)) {
    const t1 = new mime.Mime()
    const mimeType = t1.getType(imagePath) || 'image/png'
    res.setHeader('content-type', mimeType)
    return res.sendFile(imagePath)
  } else {
    return res.status(404).json({})
  }
})
```

=====